41.3 TUI 高级特性

> **生成模型**：GPT-5.4（openai/gpt-5.4）
> **Token 消耗**：输入 ~35k tokens，输出 ~3.8k tokens（本节）

这一节不再讲骨架和事件流，而是看 TUI 里那些更“工程化”的细节：文本怎么清洗，Markdown 怎么落到终端里，工具状态怎么显示，主题怎么自适应，搜索为什么手感不错。这些机制单看都不大，合起来却决定了终端界面的上限。

41.3.1 文本清理与安全渲染

终端面对的是原始字节流：ANSI 转义码、控制字符、混杂的二进制内容、超长 token，甚至还有 **从右到左文本**（Right-to-Left，RTL）带来的显示错位。OpenClaw 用 `sanitizeRenderableText()` 把这些问题串成一条清理流水线。

第一步是去掉 **ANSI 转义码**（ANSI escape sequence）。像 `\x1b[31m` 这种序列本来用于改颜色、移动光标，但如果把外部工具原样输出到 TUI，就可能污染布局，甚至伪造界面状态。所以代码先调用 `stripAnsi()` 做正则剥离。

第二步是删除 **控制字符**（control character），覆盖 ASCII 控制区和 C1 控制区。这里保留了制表符、换行和回车，因为它们对排版有意义；其余字符则直接过滤。

```
export function sanitizeRenderableText(text: string): string {
  const withoutAnsi = text.includes("\u001b") ? stripAnsi(text) : text;
  const withoutControlChars = hasControlChars(withoutAnsi)
    ? stripControlChars(withoutAnsi)
    : withoutAnsi;

  const redacted = withoutControlChars
    .split("\n")
    .map((line) => redactBinaryLikeLine(line))
    .join("\n");

  const tokenSafe = redacted.replace(LONG_TOKEN_RE, normalizeLongTokenForDisplay);
  return applyRtlIsolation(tokenSafe);
}
```

第三步处理 **二进制数据伪装**。如果一行里替换字符 `\uFFFD` 达到 12 个以上，而且占比很高，就直接替换成 `[binary data omitted]` 。这一步的目标不是保真，而是避免终端被乱码拖垮。

第四步是超长 token 切块。连续超过 32 个字符的片段会破坏换行，但 OpenClaw 也不会一律插空格，而是先判断它是不是 **复制敏感内容** （copy-sensitive token）： URL、文件路径、类似凭证的字符串、带下划线的文件名。这些内容一旦被改写，用户复制回去就失效，所以 `normalizeLongTokenForDisplay()` 只切普通长串，敏感 token 原样保留。

最后一步是 **RTL 隔离** （RTL isolation）。阿拉伯文、希伯来文这类文字如果直接和英文混排，终端可能把标点、括号和路径顺序显示错。OpenClaw 在检测到相关 Unicode 区间后，会用 `\u2067` 和 `\u2069` 包住整行，告诉终端把这段文字当作独立的 RTL 上下文处理。

41.3.2 Markdown 渲染管线

OpenClaw 没有自己重写 Markdown 解释器，而是把 `@mariozechner/pi-tui` 的 Markdown 组件包了一层，形成 `MarkdownMessageComponent` 。基础解析交给底层库，OpenClaw 主要补链接、主题和代码高亮。

```
export class MarkdownMessageComponent extends Container {
  private body: Markdown;

  constructor(text: string, y: number, options?: MarkdownOptions) {
    super();
    this.body = new Markdown(text, 1, y, markdownTheme, options);
    this.addChild(new Spacer(1));
    this.addChild(this.body);
  }
}
```

其中一个关键补丁是 `HyperlinkMarkdown` 。普通终端里，长 URL 一旦换行，点击能力很容易丢。OpenClaw 通过 **OSC 8 超链接协议**（OSC 8 hyperlink protocol）把文本和真实 URL 绑定起来，格式大致如下：

```
\x1b]8;;url\x1b\\text\x1b]8;;\x1b\\
```

这串转义码本质上就是“把 `text` 显示成可点击的 `url` ”。用户看到的是普通文字，终端知道它背后该跳到哪里。

代码块高亮则交给 **命令行语法高亮库**（cli-highlight）。它支持一百多种语言，OpenClaw 通过 `createSyntaxTheme()` 生成高亮主题，再挂进 `markdownTheme.highlightCode` 。如果显式语言可用，就按该语言高亮；否则退回自动检测；再不行就直接使用普通代码色。

```
const syntaxTheme = createSyntaxTheme(fg(palette.code), lightMode);

export const markdownTheme: MarkdownTheme = {
  heading: (text) => chalk.bold(fg(palette.accent)(text)),
  link: (text) => fg(palette.link)(text),
  code: (text) => fg(palette.code)(text),
  quote: (text) => fg(palette.quote)(text),
  highlightCode,
};
```

所以 Markdown 在这里不是“解析完就结束”，而是一条完整渲染管线：原始文本先清理，再进入 pi-tui 的结构化渲染，最后由 `markdownTheme` 把语义节点映射到具体的 `chalk` 格式器。

41.3.3 工具执行可视化

OpenClaw 的 TUI 不只是聊天窗口，它还要把 Agent 调工具的过程展示出来。 `ToolExecutionComponent` 负责这个任务，核心状态只有三种：执行中用黄色系 `toolPendingBg` ，成功用绿色系 `toolSuccessBg` ，失败用红色系 `toolErrorBg` 。颜色背后的语义非常直接，用户扫一眼就知道系统现在卡在哪一步。

```
const bg = this.isPartial
  ? theme.toolPendingBg
  : this.isError
    ? theme.toolErrorBg
    : theme.toolSuccessBg;

this.box.setBgFn((line) => bg(line));
```

更关键的是展开与折叠。工具输出经常很长，特别是 `bash` 、网页抓取和 JSON 结果。如果全部铺开，聊天记录会被瞬间冲垮；如果只显示标题，信息又不够。所以组件默认只给 12 行预览，超出部分显示省略号，用户再决定要不要展开。

`ChatLog` 内部用 `chatLog.toolById` 这个 `Map` 跟踪活跃工具组件。工具开始执行时建立条目，参数更新时复用原组件，结果返回时再把状态从 partial 切到 final。这样既减少了组件重建，也让同一个 `toolCallId` 的所有变化都落在同一块区域里。

工具结果也分层展示。 `text` 类型直接进入 Markdown 区； `image` 类型不在终端里强行渲染，而是显示 MIME 类型和大小，比如 `[image/png 128kb]` ； `details` 则保留结构化元数据，供后续展示逻辑使用。

41.3.4 亮暗主题自适应

TUI 最尴尬的问题之一是：程序并不知道用户终端到底是浅底还是深底。OpenClaw 的 `isLightBackground()` 用了一个三层判断。

第一层看 `OPENCLAW_THEME` 环境变量。这个优先级最高，因为它代表用户显式指定。第二层看 `COLORFGBG` ，这是很多 xterm 兼容终端会提供的前景色/背景色提示。第三层才是颜色学计算：把背景色转换成 **相对亮度**（relative luminance），再比较 `DARK_TEXT` （ `#E8E3D5` ）和 `LIGHT_TEXT` （ `#1E1E1E` ）谁的 **对比度**（contrast ratio）更高。

```
function isLightBackground(): boolean {
  const explicit = process.env.OPENCLAW_THEME?.toLowerCase();
  if (explicit === "light") return true;
  if (explicit === "dark") return false;

  const colorfgbg = process.env.COLORFGBG;
  // ... 解析 xterm 颜色编号
  return pickHigherContrastText(rVal, gVal, bVal);
}
```

这里提到的 WCAG，并不是 Web 才关心的规范。**网页内容无障碍指南**（Web Content Accessibility Guidelines，WCAG）本质上回答的是：文字和背景之间，怎样才算“看得清”。OpenClaw 把这套思路借到终端里，因此 `darkPalette` 和 `lightPalette` 不是简单反色，而是重新选了一组更稳妥的颜色。

主题一旦确定，后续链条会自动跟上： `palette` 决定普通文本、引用、代码块、工具状态色； `markdownTheme` 决定 Markdown 各语义节点的显示； `editorTheme` 再从同一套调色板派生输入框边框和选择列表样式。也就是说，自适应并不是一个 if/else 小开关，而是一整套派生系统。

41.3.5 模糊搜索与选择器

在模型选择、会话切换、Agent 选择这些场景里，用户通常记得一部分名字，但未必记得全称。 `SearchableSelectList` 的实现很像一个轻量版命令面板，它不是单纯“搜到就行”，而是先做优先级排序。

OpenClaw 把匹配分成四层。第 0 层是标签里的精确子串匹配，谁出现得更早，谁排前面；第 1 层是单词边界前缀匹配，比如在 `/` 、 `-` 、空格之后开始命中；第 2 层看描述字段中的精确子串；第 3 层才退到模糊匹配。这个顺序基本贴合了用户真实意图。

```
private smartFilter(query: string): SelectItem[] {
  // Tier 0: label exact substring
  // Tier 1: word-boundary prefix
  // Tier 2: description exact substring
  // Tier 3: fuzzy match
}
```

底层的 `fuzzyMatchLower()` 没有走复杂的编辑距离算法，而是用了更轻的评分模型：跳过字符有惩罚，间隔越大惩罚越多；连续命中会奖励；命中单词边界再额外加分。如果完全匹配不上，直接返回 `null` 。

另一个容易被忽略的点是 ANSI 处理。选择器里的 label 和 description 可能带颜色，如果直接拿带 ANSI 的字符串做搜索，用户输入 `32m` 这种片段都可能误命中。OpenClaw 一边在搜索前做 ANSI stripping，一边把正则缓存进 `regexCache` ，避免用户每敲一个字符都重复构造对象。

41.3.6 性能优化策略

终端程序很容易给人一种“反正就是打印文本，性能不是问题”的错觉。其实恰恰相反：TUI 越接近实时界面，越要控制增量更新和状态膨胀。OpenClaw 在这里做了几层很朴素、但很有效的优化。

先看 `ChatLog` 。它不是无限堆消息，而是把组件缓冲区限制在 180 个左右，溢出后清理最旧项，并同步删除相关引用。这样能把渲染成本稳定住。

运行态追踪也有限额。 `tui-event-handlers.ts` 里对活跃 run 和已完成 run 都使用 `Map` 记录时间戳，数量超过 200 后就开始修剪；优先删除超过 10 分钟 TTL 的旧项，如果还超限，再继续裁到 150 左右。

```
if (runs.size <= 200) return;
const keepUntil = Date.now() - 10 * 60 * 1000;
for (const [key, ts] of runs) {
  if (runs.size <= 150) break;
  if (ts < keepUntil) runs.delete(key);
}
```

渲染层面则尽量做 **增量更新**（incremental rendering）。流式输出进入 `TuiStreamAssembler` 后，会比较新的 `displayText` 和旧值；如果文本没变， `ingestDelta()` 直接返回 `null` ，上层也就不触发无意义重绘。很多模型流会重复带上已有内容，这一步正好用来去重。

最后，模糊搜索里的正则缓存、预先 lower-case 的搜索文本、流组装阶段的 `displayText` 去重，本质上都属于同一种思路：不要反复做已经做过的事。TUI 的性能优化通常没有什么“神之一手”，更多是把小成本一个个抠掉。

本节小结

这一节真正想说明的是：TUI 的高级特性不只是“多几个颜色、多几个组件”。OpenClaw 在文本清理、Markdown 渲染、工具执行展示、主题自适应、模糊搜索和性能控制上，做的都是终端环境里很具体的问题求解。它没有把终端硬拗成 GUI，而是承认终端的限制，再围绕这些限制做出合适的工程折中。